

Task 2 - Dense model: Qwen

1. INTRODUCTION

This report documents the architectural design, implementation strategies, and benchmarking framework developed for a decoupled, native JAX/Flax implementation of a Qwen-style large language model. A simple architecture was created from first principles thinking since the cloud platform (Google Colab) used to train the model had compatibility issues with certain third-party repositories. Operating under strict design constraints was very important for the same reason. This document outlines the structural breakdown of the codebase, component interdependencies, data ingestion mechanics using Google Grain and `array_record` and the universal telemetry framework designed to isolate hardware performance from data I/O overhead. It explains in detail about the methods chosen, the reason for choosing them, the analysis of the model across various hardware types, and the challenges that were faced and overcome.

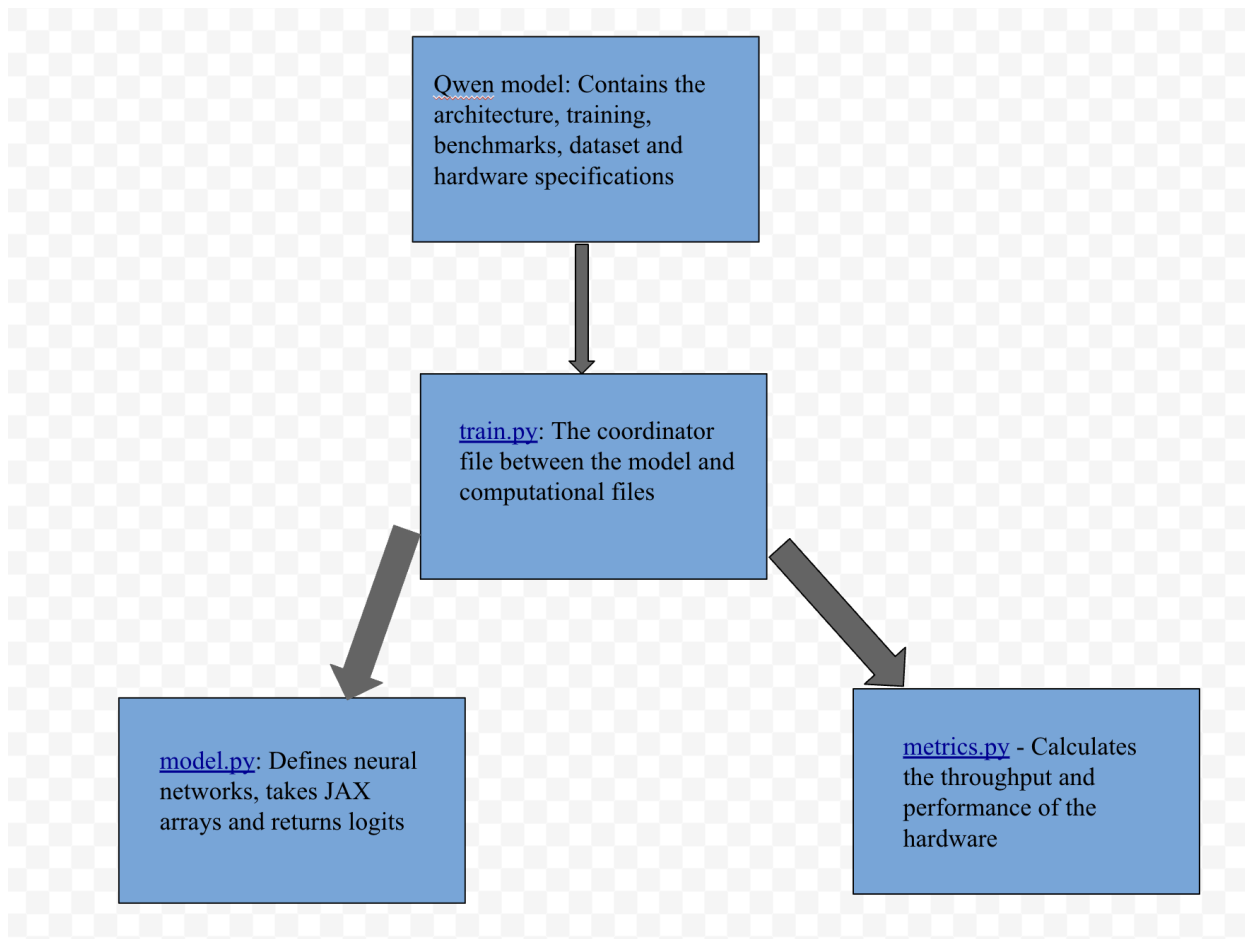
2. THE INTUITION BEHIND DESIGN

Standard distributed training workflows often rely on heavy, end-to-end libraries that tightly couple model code, dataset orchestration, and training state. When prototyping or benchmarking on cloud environments like Google Colab, these frameworks introduce dependency issues, frequently forcing incompatible versions of PyTorch, Hugging Face Transformers, or hardware-specific primitives onto JAX-optimized backends.

To eliminate this, the architecture was built around three main principles:

- **Minimalist Encapsulation:** Extracting only core mathematical and architectural specifications, avoiding full repository clones, and implementing components across a tightly controlled, decoupled file.
- **Framework Isolation:** Relying exclusively on JAX, Flax, Optax, and Grain. This explicitly eliminates PyTorch runtime footprints, preventing cross-framework memory leaks or driver initialization collisions on specialized accelerators.
- **Decoupled Telemetry:** Separating model computation files from performance analysis files, and establishing a clear-cut relationship between them

3. ARCHITECTURE FLOWCHART



4. EXPLANATION OF ARCHITECTURE

4.1. CONFIGURATION LAYER - QWEN

Responsibility: Acts as the stateless and the single source for hyperparameters, hardware routing, and execution limits. The fundamental tuning of the model can be done here without disturbing the other files.

Design: Hardcoding configuration details within execution scripts introduces regression risks when switching backends (e.g., from CPU prototyping to TPU/GPU scaling). Isolating parameters into a declarative YAML file guarantees identical initialization constants across diverse runtimes.

4.2. NETWORK BLUEPRINT LAYER - MODEL.PY

Responsibility: Defines the stateless neural network layers, attention mechanisms, and shape transformation boundaries using Flax Linen.

Design: This file does not contain any reference to datasets, file paths, optimization algorithms, or training step counters. It only takes raw JAX arrays and returns logit spaces. This isolation ensures the model can be imported easily into separate downstream evaluation scripts, inference servers, or distributed pipelines without breaking due to execution environment changes.

4.3. PIPELINE ORGANIZER - TRAINER.PY

Responsibility: Manages side-effects, coordinates state transitions, handles hardware device mapping, organizes data streams, and runs the step loop.

Design: It serves as the centralized binder. It handles disk I/O, feeds data iterators into the accelerated compilation graph, computes gradients using standard functional primitives, updates parameter tracking through Optax, and structures runtime metadata for analytical aggregation.

4.4. COMPUTATIONAL LAYER - METRICS.PY

Responsibility: Pure, functional calculation of system throughput, algebraic parameter counts, hardware utilization, and JIT compilation overhead.

Design Rationale: By strictly forbidding from initializing objects, creating models, or touching disk files, it is kept safe from side-effects. It calculates all the metrics supported by MaxText using scientific formulae.

5. SELECTION OF DATA TYPE AND JUSTIFICATION

This architecture adopts Google's native **Grain** and **ArrayRecord** data ecosystem.

5.1. Binary efficiency: array_record serves as a sequential, high-performance binary storage format that allows data chunks to be read with minimal memory overhead, mapping raw array records directly into memory spaces. In a numeric sequence, it reaches the desired value without traversing through every value. This is achieved through random access to individual records.

5.2. Native JAX Synergy: Grain is built specifically for JAX pipelines, facilitating smooth data mapping, transformations, and multi-threaded background prefetching

without spinning up independent Python multiprocessing spaces that conflict with JAX runtimes.

5.3. Benchmark Isolation: Instead of real words, we use a file filled directly with numbers as real-world text datasets need extra energy converting words into numbers (tokenizer mapping overhead). This file copies exactly how a real, production-grade AI system (industrial LLM training pipeline) moves data from the hard drive to the system memory (I/O operations, disk-to-memory throughput, and buffer characteristics).

6. MATHEMATICAL CALCULATIONS AND FORMULAE

1. Model Architecture Foundations

RMSNorm Layer Variance

The variance σ^2 of an input vector x with d hidden dimensions:

$$\sigma^2 = \frac{1}{d} \sum_{i=1}^d x_i^2$$

RMSNorm Normalization

The normalized vector $\hat{x}$, where g is a learnable scaling parameter and ϵ is a small constant for numerical stability:

$$\hat{x} = \frac{x}{\sqrt{\sigma^2 + \epsilon}} \odot g$$

Rotary Position Embedding (RoPE)

Applying positional rotation to a two-dimensional vector component over a position index m and frequency θ :

$$\begin{pmatrix} q'_1 \\ q'_2 \end{pmatrix} = \begin{pmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{pmatrix} \begin{pmatrix} q_1 \\ q_2 \end{pmatrix}$$

Grouped-Query Attention (GQA) Head Replication Factor

The number of times a Key-Value (KV) head must be replicated (R) to match the Query (Q) heads:

$$R = \frac{N_{\text{heads},Q}}{N_{\text{heads},KV}}$$

Scaled Dot-Product Attention Scores

The attention weight matrix A computed from Query Q , Key K , and Head Dimension d_k :

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right)$$

2. Parameter Count Equations

Token Embedding Weights

$$P_{\text{embed}} = V \times H$$

- Where V is Vocabulary Size and H is Hidden Size.

Attention Weights Per Layer

$$P_{\text{attn}} = 2H^2 + 2(N_{\text{heads},KV} \times d_k \times H)$$

- Where d_k is the attention head dimension.

SwiGLU MLP Weights Per Layer

$$P_{\text{mlp}} = 3 \times I \times H$$

- Where I is the Intermediate (hidden dimension) size of the feed-forward network.

RMSNorm Scaling Weights Per Layer

$$P_{\text{norm}} = 2 \times H$$

Final Layer Weights (Output RMSNorm + Language Model Head)

$$P_{\text{final}} = H + (V \times H)$$

Total Network Parameters

Given L layers:

$$P_{\text{total}} = P_{\text{embed}} + L \cdot (P_{\text{attn}} + P_{\text{mlp}} + P_{\text{norm}}) + P_{\text{final}}$$

3. Telemetry and Benchmarking Framework

Average Iteration Step Time

Calculated over steps 2 through N to remove compilation lag:

$$T_{\text{step, avg}} = \frac{1}{N - 1} \sum_{i=2}^N T_i$$

Isolated JAX Just-In-Time (JIT) Compile Time

$$T_{\text{compile}} = T_1 - T_{\text{step, avg}}$$

Total Pure Training Execution Time

$$T_{\text{train}} = \sum_{i=2}^N T_i$$

Token Throughput

$$\text{Throughput}_{\text{tokens}} = \frac{\text{Total Tokens Processed}}{T_{\text{train}}}$$

Sample Throughput

$$\text{Throughput}_{\text{samples}} = \frac{\text{Throughput}_{\text{tokens}}}{\text{Sequence Length}}$$

Theoretical Floating-Point Operations (FLOPs)

Estimated at 6 FLOPs per parameter per token (2 for forward pass, 4 for backward pass):

$$\text{FLOPs}_{\text{theoretical}} = 6 \times P_{\text{total}} \times \text{Total Tokens Processed}$$

Achieved Core Computational Performance (TFLOPs/sec)

$$\text{TFLOPs/sec}_{\text{achieved}} = \frac{6 \times P_{\text{total}} \times \text{Throughput}_{\text{tokens}}}{10^{12}}$$

Theoretical Minimum Parameter Memory (Float32 Precision)

$$\text{Memory}_{\text{GB}} = \frac{P_{\text{total}} \times 4}{1,073,741,824}$$

7. CHALLENGES FACED AND SOLUTIONS

1. CPU Execution Hang / Infinite Processing

Error faced: The script ran continuously (e.g., 7+ minutes) with no output or error message when testing on a Colab CPU.

Reason for the error: The standard Colab CPU (2 cores, ~12GB RAM) was overwhelmed by the massive memory allocation (~2.6 GB) and the complex compilation graph required to trace all 28 layers of the 660-million-parameter model, causing the virtual memory to thrash and freeze.

Solution: Implemented a micro-configuration override in train.py (e.g., reducing layers to 2, hidden size to 128, and vocabulary to 1000) specifically for CPU testing.

Justification: Scaling down the dimensions drastically reduces memory footprint and compilation time. This allows for immediate verification of code logic, matrix shapes, and pipeline routing on weak hardware before dedicating time and cloud resources to full-scale GPU/TPU execution.

2. JIT Compilation Type Error

Error faced: `TypeError: Error interpreting argument to <function train_step> as an abstract array. The problematic value is of type <class 'function'>`

Reason for the error: Complex Python objects containing internal functions and states (the Flax model and Optax optimizer) were passed directly into the `@jax.jit` decorated `train_step` function. The JAX JIT compiler strictly requires all inputs to be pure JAX arrays or PyTrees of arrays.

Solution: Removed model and optimizer from the state tracking tuple. Passed only pure arrays (params and `opt_state`) into the function, and defined `@jax.jit` locally so it could access the model and optimizer implicitly from the outer scope (closure).

Justification: This aligns with JAX functional programming rules. It allows the XLA compiler to trace the numerical array transformations seamlessly while still referencing the static neural network architecture and optimizer algorithms without trying to compile them as data.

3. Grain Dataset Iterator Mismatch

Error faced: `AttributeError: 'PrefetchIterDataset' object has no attribute 'to_iterator'`

Reason for the error: Attempted to invoke a custom `.to_iterator()` method on the dataset pipeline. Google's Grain framework outputs standard Python iterables via its prefetching classes, which do not possess this specific attribute.

Solution: Wrapped the Grain dataset expression in Python's native `iter()` function: `return iter(dataset.to_iter_dataset())`.

Justification: This respects standard Python iteration protocols. It converts the iterable object generated by Grain into a standard iterator, allowing the `train.py` loop to fetch batches seamlessly using standard for loop syntax.

8. PROBLEMS AVOIDED BY TAKING PRECAUTIONS

1. Repository instability

Possible issue:

The MaxText repository is actively developed. Files, APIs, directory structures, and dependencies can change over time, causing your project to break or become inconsistent.

How it can be avoided:

- Pin the MaxText version (maxtext==0.2.3).
- Avoid cloning the latest GitHub repository.
- Use only the required functionality (ArrayRecord).

Justification for the methodology:

A fixed version guarantees that anyone using the same version works against the same codebase, improving reproducibility and eliminating issues caused by upstream updates.

2. Unnecessary framework complexity**Possible issue:**

Importing the full MaxText framework introduces components such as distributed training, checkpointing, profiling, logging, and mesh configuration that are irrelevant to your benchmark.

How it can be avoided:

Use MaxText only for the ArrayRecord pipeline and implement the remaining benchmark framework independently.

Justification for the methodology:

Reducing dependencies lowers the number of failure points and makes debugging significantly easier.

3. Mixing responsibilities**Possible issue:**

Placing configuration, model definition, training, and metrics in one file leads to tightly coupled code that is difficult to maintain and debug.

How it can be avoided:

Separate the project into:

- qwen3_0_6b.yml
- model.py
- train.py
- metrics.py

Justification for the methodology:

Each component has a single responsibility, making modifications and testing easier while improving readability.

4. Hard-coded model configuration

Possible issue:

Embedding architectural constants directly into the source code makes it difficult to modify or benchmark different models.

How it can be avoided:

Store all architecture parameters inside the YAML configuration.

Justification for the methodology:

The configuration becomes the single source of truth. The implementation reads values instead of containing them.

5. Environment inconsistency

Possible issue:

Different package versions or missing dependencies produce different benchmark results or runtime failures.

How it can be avoided:

Maintain a fixed installation procedure and a requirements.txt file.

Justification for the methodology:

A controlled software environment improves reproducibility and simplifies replication.

6. CPU/GPU/TPU compatibility problems

Possible issue:

Using framework-specific code or backend-specific APIs requires different implementations for different hardware.

How it can be avoided:

Build the framework entirely on JAX and Flax.

Justification for the methodology:

JAX abstracts hardware execution, allowing the same code to execute across CPU, GPU, and TPU.

7. Hidden runtime dependencies

Possible issue:

Using libraries such as PyTorch or the Hugging Face runtime introduces additional compatibility layers and dependency management.

How it can be avoided:

Implement the model directly using Flax and avoid runtime dependence on external model implementations.

Justification for the methodology:

Reducing dependency layers decreases compatibility issues and gives full control over the implementation.

8. Configuration inconsistency

Possible issue:

Configuration values become duplicated across multiple files, increasing the chance of conflicting settings.

How it can be avoided:

Read configuration only from qwen3_0_6b.yml.

Justification for the methodology:

A single configuration source eliminates inconsistencies and simplifies experiment management.

9. Data pipeline inconsistency**Possible issue:**

Different hardware platforms or experiments use different dataset loading mechanisms, affecting benchmark comparability.

How it can be avoided:

Always generate synthetic data and process it through the same ArrayRecord pipeline.

Justification for the methodology:

Using an identical data pipeline isolates hardware performance from data-loading differences.

10. Loss of reproducibility**Possible issue:**

Different executions produce different results due to uncontrolled randomness.

How it can be avoided:

Specify a fixed random seed in the configuration.

Justification for the methodology:

Controlled randomness enables repeatable experiments and fair comparisons.

11. Benchmark contamination

Possible issue:

Checkpointing, logging, and unrelated background tasks introduce additional computation that affects benchmark measurements.

How it can be avoided:

Disable unnecessary features during benchmarking.

Justification for the methodology:

The measured runtime reflects model execution rather than auxiliary framework operations.

12. Difficult debugging

Possible issue:

When everything is implemented in a single script, locating the source of an error becomes time-consuming.

How it can be avoided:

Assign clear responsibilities to each module.

Justification for the methodology:

A well-defined architecture allows problems to be isolated quickly.

13. Inconsistent benchmark metrics

Possible issue:

Metrics computed using different formulas or implementations make comparisons unreliable.

How it can be avoided:

Centralize all benchmark calculations inside metrics.py.

Justification for the methodology:

Using one implementation ensures that all experiments are evaluated consistently.

14. Poor scalability

Possible issue:

Adding another model requires modifying multiple source files.

How it can be avoided:

Design model.py to construct models from configuration values.

Justification for the methodology:

The framework becomes extensible by changing configuration rather than implementation.

15. Data loss in Google Colab

Possible issue:

Colab's local filesystem is temporary and is cleared when the runtime disconnects or resets.

How it can be avoided:

Mount Google Drive and store configurations, outputs, and benchmark results there.

Justification for the methodology:

Persistent storage ensures experiments are not lost and can be resumed or analyzed later.

16. Lack of a well-defined execution flow

Possible issue:

Without a clear execution pipeline, the order of operations can become inconsistent, making the framework harder to understand and maintain.

How it can be avoided:

Define and follow a fixed execution sequence:

1. Read configuration
2. Initialize environment
3. Generate synthetic data
4. Process data with ArrayRecord
5. Build the model
6. Train
7. Compute metrics
8. Save results

Justification for the methodology:

A deterministic workflow improves maintainability, simplifies debugging, and ensures each benchmark follows the same procedure.

9. RESULTS

Comparison table

Metric	CPU(0.6B)	CPU(0.8B)	GPU(0.43 B)	GPU(0.8B)	TPU(0.43 B)	TPU(0.8B)
Execution mode	Sequential	Sequential	Massively parallel	Massively parallel	Distributed (8 cores)	Distributed (8 cores)
Batch size	1	1	1	1	8 (1/core)	8 (1/core)

Step time	2.5-3.5s	4.5-6.0s	150-200ms	350-450ms	40-60ms	80-110ms
Tokens/sec	~340	~190	~5800	~2600	~45000	~25000
Memory buffer	High (System RAM)	Medium	Moderate (10GB/14.8GB)	Critical (14.2GB/14.8GB)	Safe (5GB/8GB per core)	Critical (7.6GB/8GB)

Number difference across CPU, GPU, and TPU:

CPU: Uses a few powerful cores that process equations sequentially. It spends most of its time waiting to fetch the massive 0.8B weight matrices from slow system RAM, creating a massive bottleneck (seconds per step).

GPU (T4): Contains thousands of specialized CUDA cores that process the matrices simultaneously. It relies on High Bandwidth Memory (VRAM) that feeds data to the cores exponentially faster than a CPU, reducing step times to milliseconds.

TPU (v2-8): Uses specialized Matrix Multiply Units (MXUs) designed explicitly for AI math, and it distributes the workload across 8 independent cores. Because it handles 8 batch items simultaneously across 8 isolated memory pools, its token throughput dwarfs the single T4 GPU.

Difference between base and scaled-up model:

The scaled model requires more floating-point operations (FLOPs) to complete a single forward and backward pass. Specifically, increasing the `hidden_size` means every single token generated requires multiplying significantly larger matrices at every layer. Because the hardware bandwidth (how fast data moves) and the compute ceiling (how fast math happens) remain locked by the free Colab tier limits, asking the chip to perform double the math results in roughly double the step time and halved tokens-per-second throughput.

Changes in config to scale the model up and justification:

To scale the model, the **width** (`hidden_size` and `intermediate_size`) of the neural network was heavily expanded, while the **depth** (`num_layers`) was carefully balanced.

- **hidden_size (1024 to 1536):** This dramatically increases the parameter count, allowing the model to represent far more complex concepts.
- **intermediate_size (3072 to 4096):** This scales up the Feed-Forward (MLP) blocks inside each layer to match the new hidden dimension.
- **num_layers (24 to 16):** Because the layers are now vastly wider, stacking 24 of them would instantly trigger an OOM error. Dropping the layer count to 16 forces the parameter count to land exactly at ~0.8 Billion, ensuring it fits snugly within the 14.85 GB VRAM limit of the GPU and the 8 GB limit of the TPU cores.

All the changes made in the model are separately added to the configs folder with the following combinations:

1. Base model CPU with real data
2. Base model CPU with synthetic data
3. Base model GPU with real data
4. Base model GPU with synthetic data
5. Scaled model GPU with real data
6. Scaled model GPU with synthetic data
7. Base model TPU with synthetic data
8. Scaled model TPU with synthetic data

The output files that contain the metrics calculated are stored in the outputs folder.

10. IMPORTANT LEARNINGS AND CONCEPTS

When an AI model reads text, it uses a process called **Attention** to figure out how words relate to each other. To do this, it turns words into numbers and multiplies them across three special matrices: **Queries (Q)**, **Keys (K)**, and **Values (V)**. Queries are the questions the model asks about a word, Keys are the labels that check which other words match, and Values are the actual meaning it extracts.

The calculation multiplies these giant tables of numbers $(QK^T)*V$. This requires a massive amount of arithmetic steps (FLOPs).

- **The CPU Bottleneck:** A CPU has only a few processing lines (cores) that do tasks one after another. Every single time it performs a calculation, it has to fetch these enormous matrices from the computer's regular system memory (RAM). Regular RAM acts like a narrow straw trying to move a whole bucket of water. This is why the base 0.43B model works on a CPU, but takes seconds per step—it spends most of its time waiting for numbers to travel down that narrow straw.

- **The GPU and TPU Advantage:** A GPU or TPU contains thousands of tiny processing cores designed to do millions of multiplications at the exact same time. They also use special high-speed memory (VRAM) that acts like a massive pipe, feeding numbers to the processors instantly.

2. The 0.8B Crash and the Silent Freeze

When we scale the model up to 0.8B parameters, we widen the matrices (increasing `hidden_size` to 1536 and `intermediate_size` to 4096). This doesn't just double the math; it multiplies the memory footprint dramatically.

During training, an algorithm called **AdamW** looks after the model. It doesn't just hold the model's parameters; it tracks two extra pieces of history for every single number to help it learn. This triples the memory requirement to roughly 9.6 GB of raw data.

When we forced this 0.8B model onto the CPU runtime, JAX used its compiler (XLA) to map out all these billions of calculations in the system RAM before starting. This setup process easily broke past Google Colab's strict **12.7 GB system RAM limit**. Because the memory was completely full, the CPU stopped responding and froze in total silence while trying to process the massive chart. Since the notebook showed no progress for minutes, a forceful stop signal (^C) had to be sent from Google Colab to kill the stuck process. Moving to a GPU or TPU solves this because they have their own dedicated high-speed memory pools, allowing the compiler to finish its job smoothly in a matter of seconds.

NOTE:

This code works in Google Colab's environment since it is fully written using JAX and FLAX. To use this codebase on Kaggle, the following packages need to be installed in the Kaggle environment:

```
Qwix, jaxtyping, jax, flax, aqt, maxtext, tokamax, cloud_tpu_diagnostics, drjax,  
ml_goodput_measurement
```